Algorithmes et Pensée Computationnelle

Programmation orientée objet : Héritage et Polymorphisme

Le but de cette séance est d'approfondir les notions de programmation orientée objet vues précédemment. Nous commencerons par un rappel des notions de surcharge d'opérateurs avant d'introduire des exercices sur l'héritage et le polymorphisme. Au terme de cette séance, vous devez être en mesure de factoriser votre code afin de le rendre mieux structuré et plus lisible. À chaque exercice, le langage de programmation à utiliser sera spécifié.
Le code présenté dans les énoncés se trouve sur Moodle, dans le dossier `Code`.

1 Rappel : Surcharge des opérateurs - Python

Dans cette section, vous manipulerez des fractions sous forme d'objets. Vous ferez des opérations de base sur ce nouveau type d'objets.

Question 1: (🕓 *5 minutes*) **Création de classe**
Dans un projet que vous aurez au préalable préparé, créez un fichier nommé `surcharge.py`. À l'intérieur de ce fichier, créez une classe `Fraction` qui aura comme attributs privés un numérateur et un dénominateur.

Question 2: (🕓 *5 minutes*) **Constructeur par défaut**
Définissez un constructeur pour votre classe. Assignez des valeurs par défaut à vos attributs.
Si un seul argument est passé à votre constructeur, la fraction devra être égale à l'entier correspondant.
Empêchez l'utilisateur d'assigner la valeur `zéro` au dénominateur.

> **Conseil**
>
> Les valeurs par défaut seront assignées à votre objet s'il est instancié sans arguments. Ainsi, en faisant `f = Fraction()`, on obtiendra un objet `Fraction` ayant un numérateur égal à 0 et un dénominateur égal à 1, soit $\frac{0}{1}$.
> En Python, vous pouvez donner des valeurs par défaut aux arguments de vos méthodes lors de leur définition. Par exemple, vous pouvez faire : `def add(self, x=10, y=5):...`

Question 3: (🕓 *5 minutes*) **Type casting**
Convertissez les valeurs des attributs en entiers.

> **Conseil**
>
> Pensez à utiliser la fonction `int`.

Question 4: (🕓 *5 minutes*) **Redéfinition de méthodes**
Redéfinissez la méthode `__str__()` pour produire une représentation textuelle de vos objets `Fraction`.

> **Conseil**
>
> Une fois la méthode `__str__()` redéfinie, lorsqu'on appellera `print()` sur une instance de votre classe `Fraction`, cet appel affichera le message suivant : *Votre fraction a pour valeur `numérateur/dénominateur`*. Le numérateur et le dénominateur correspondent aux valeurs des attributs de votre objet `Fraction`.

Question 5: (🕓 *5 minutes*) **Accesseurs et mutateurs**
Créez des `getters` et `setters` pour chacun des attributs de votre classe `Fraction`.

Question 6: (🕓 *10 minutes*) **Simplification de fractions**
Définissez une méthode `simplification` qui réduit la `Fraction`. Cette méthode ne renverra rien, elle

modifiera simplement l'instance. Pour la suite des exercices, assurez-vous de toujours manipuler des fractions simplifiées. Pour ce faire, vous pouvez faire appel à votre méthode `simplification` après chaque opération sur un objet de type `Fraction`.

Conseil

Afin de simplifier une fraction, vous devez diviser le numérateur et le dénominateur par leur plus grand diviseur commun. Pensez à utiliser la méthode `math.gcd` pour trouver le plus grand diviseur commun entre deux nombres. Si le dénominateur d'une fraction est négatif, on peut changer le signe du numérateur et du dénominateur pour rendre le dénominateur positif.

Question 7: (*15 minutes*) **Redéfinition de méthodes - __eq__**
Redéfinissez la méthode d'instance `__eq__` qui prend en entrée un objet `Fraction` que vous nommerez *other* (en plus de `self`) et qui renvoie `True` si `self` et l'objet passé en argument ont la même valeur.

Conseil

Pour vérifier l'égalité entre a/b et c/d, testez que $a * d$ est égal à $b * c$.
Utilisez la méthode `isinstance()` afin de vérifier que *other* est bien de type `Fraction`. Dans le cas contraire, affichez un message d'erreur.
La fonction `isinstance` prend en entrée une valeur et un type. Elle vérifie que cette valeur est du type défini. Par exemple : `isinstance(nombre, int)` renverra `True` si la variable `nombre` est de type `int` et `False` dans le cas contraire.

Question 8: (*15 minutes*) **Addition et multiplication**
Redéfinissez les méthodes `__add__` et `__mul__` afin d'effectuer des opérations d'addition et de multiplication sur vos objets de type `Fraction`. Attention, ces méthodes devront renvoyer des objets de type `Fraction`. Dans vos méthodes `__add__` et `__mul__`, n'oubliez pas de simplifier ces fractions avant de les retourner.
Gérez le cas où l'élément passé en argument n'est ni une `Fraction` ni un `int`.

Solution

```python
# Question 1: Création de la classe Fraction
class Fraction:
    # Question 2: Déclaration du constructeur et initialisation des valeurs
    def __init__(self, numerateur=0, denominateur=1):
        # Question 3: type casting
        self.__num = int(numerateur)
        if int(denominateur) == 0:
            raise ZeroDivisionError("Le dénominateur ne peut pas être égal à zéro")
        self.__den = int(denominateur)
        self.simplification()

    # Question 4: redéfinition de la méthode __str__()
    def __str__(self):
        return "Votre fraction a pour valeur {}/{}".format(self.__num, self.__den)

    # Question 5: Getters et Setters
    def get_num(self):
        return self.__num

    def get_den(self):
        return self.__den

    def set_num(self, n):
        self.__num = n

    def set_den(self, d):
        d = int(d)
        # Si on passe 0 au dénominateur, on lève une exception ce qui arrêtera le programme
        if d == 0:
            raise ZeroDivisionError("Le dénominateur ne peut pas être égal à zéro")
        self.__den = d

    # Question 6
    def simplification(self):
        if self.__num == 0:
            self.__den = 1
        if self.__den < 0:
            self.__num = -self.__num
            self.__den = -self.__den
        pgcd = math.gcd(self.__num, self.__den)
        self.__num = int(self.__num / pgcd)
        self.__den = int(self.__den / pgcd)

    # Question 7
    def __eq__(self, f):
        if isinstance(f, Fraction):
            # vu que les fractions sont toujours en représentation simplifiée, on pourrait se contenter de
            # self.__numerateur == f.__numerateur and self.__denominateur = f.denominateur
            return self.__num * f.__den == f.__num * self.__den
        # Au cas où on reçoit un seul argument, on créé une fraction ayant pour numérateur l'argument et 1 comme dénominateur
```

Solution

```
51          elif isinstance(f, int):
52              return self.__eq__(Fraction(f))
53          else:
54              return False
```

Solution

```
# Question 8
    def add(self, f):
        self.__num = self.__num * f.__den + f.__num * self.__den
        self.__den = self.__den * f.__den
        self.simplification()

    def plus(self, f):
        q = Fraction(self.__num, self.__den)
        q.add(f)
        return q

    def __add__(self, other):
        if isinstance(other, Fraction):
            return self.plus(other)
        elif isinstance(other, int):
            self.add__ = self.__add__(Fraction(other))
            return self.add__
        else:
            print("Unsupported operand types for +: '" +
    self.__class__.__name__ + "' and '" + other.__class__.__name__ +
    "'")

    def __mul__(self, other):
        if isinstance(other, Fraction):
            return Fraction(self.__num * other.__num, self.__den *
    other.__den)
        elif isinstance(other, int):
            return Fraction(self.__num * other, self.__den)
        # On affiche un message d'erreur lorsque other n'est pas une
    Fraction
        else:
            print("Unsupported operand types for *: '" +
    self.__class__.__name__ + "' and '" + other.__class__.__name__ +
    "'")

if __name__ == '__main__':
    f1 = Fraction()
    print(f1)
    f1 = Fraction(4)
    print(f1)
    f1=Fraction(numerateur=5, denominateur=0)
    print(f1)
    f1 = Fraction(denominateur=5)
    print(f1)
    f1 = Fraction(4, -6)
    print(f1)
    f2 = Fraction(2, -8)
    print(f2)
    print(f1+f2)
    print(f1 == Fraction(22, -24))
    print(f1 == Fraction(1, 2))
```

2 Polymorphisme - Java

Les exercices de cette section s'inspirent des exercices de la section 1 des exercices avancés de la semaine 6. N'hésitez pas à réutiliser les solutions disponibles sur Moodle.
Dans cette partie, vous devez créer deux nouvelles classes filles de la classe-mère `Fighter`. La première classe représentera un `Soigneur`, qui, lorsqu'il "attaquera" un `Fighter`, le soignera au lieu de le blesser.

La deuxième classe, nommée `Attaquant`, représentera un `Fighter` spécialisé dans l'attaque et aura la capacité d'attaquer un certain nombre de fois (ce nombre sera défini lors de l'instanciation).

Avant d'effectuer la série de questions suivantes, téléchargez le fichier `Fighter.java` disponible sur Moodle.

Question 9: (*5 minutes*) classe-fille `Soigneur`
Créez une nouvelle classe-fille `Soigneur` qui hérite de la classe `Fighter`. Cette classe-fille aura un nouvel attribut privé nommé `resurrection` qui vaudra 1 lors de l'instanciation de la classe.

Créez le constructeur de cette classe ainsi que le `getter` et le `setter` permettant d'interagir avec le nouvel attribut (`resurrection`).

Conseil

Pensez à utiliser le constructeur de votre classe-mère `Fighter`.

Solution

```
1  class Soigneur extends Fighter {
2
3      private int resurrection;
4
5      public Soigneur(String name, int health, int attack, int defense) {
6          super(name, health, attack, defense);
7          this.resurrection = 1;
8      }
9
10     public int getResurrection() {
11         return this.resurrection;
12     }
13
14     public void setResurrection(int etat) {
15         this.resurrection = etat;
16     }
17 }
```

Question 10: (*15 minutes*) Méthode `resurrection(Fighter other)` de la classe-fille `Soigneur`
Créez une méthode `resurrection(Fighter other)` qui permettra de faire revenir un `Fighter` à la vie, mais le `Soigneur` ne pourra le faire qu'une seule fois.

Commencez par contrôler que l'instance depuis laquelle la méthode est appelée est toujours en vie. Si ce n'est pas le cas, indiquez : `nom_instance` est mort et ne peut plus rien faire.

Contrôlez ensuite que l'instance `other` est vraiment morte. Si ce n'est pas le cas, indiquez-le en affichant le texte suivant : "`nom_other` est toujours en vie."

Pour finir, contrôlez que l'attribut `resurrection` de l'instance depuis laquelle la méthode est appelée est égal à 1. Si ce n'est pas le cas, indiquez : `nom_instance` ne peut plus ressusciter personne.

Si tous ces éléments sont réunis, faites revenir le `Fighter other` à la vie en lui remettant 10 points de vie et en l'ajoutant à la liste `instances` de la classe `Fighter`. Pensez aussi à :

— mettre l'attribut `resurrection` de l'instance appelée à 0 afin de l'empêcher de réutiliser ce pouvoir,

— appeler la méthode `checkHealth()` et indiquer : "`nom_other` est revenu à la vie !"

Conseil

Utilisez un branchement conditionnel pour les contrôles.

Une nouvelle méthode nommée `addInstances(Fighter other)` a été créée dans la classe `Fighter`. Regardez à quoi elle sert et utilisez-la.

Pour les indications en fonction des différentes conditions, affichez simplement la phrase en question.

Solution

```
1  public void resurrection(Fighter other) {
2      if (!this.isAlive()) {
3          System.out.println(this.getName() + " est mort et ne peut plus
       rien faire");
4      } else {
5          if (other.isAlive()) {
6              System.out.println(other.getName() + " est toujours en vie
       !");
7          } else {
8              if (this.getResurrection() == 0) {
9                  System.out.println(this.getName() + " ne peut plus
       ressusciter personne");
10             } else {
11                 other.setHealth(10);
12                 Fighter.addInstances(other);
13                 this.setResurrection(0);
14                 System.out.println(other.getName() + " vient de
       revenir à la vie");
15                 Fighter.checkHealth();
16             }
17         }
18     }
19 }
```

Question 11: (*10 minutes*) Méthode `attack` de la classe-fille `Soigneur`

Réécrivez la méthode `attack` de la classe-fille `Soigneur` afin d'ajouter des points de vie à `other` au lieu de lui en retirer.

Le seul argument nécessaire pour cette méthode sera une autre instance de `Fighter` qu'on pourrait nommer `other`.

Commencez par contrôler que le `Soigneur` depuis lequel la méthode est appelée est encore en vie. Si ce n'est pas le cas, affichez : "`nom_instance` est mort et ne peut plus rien faire."
Contrôlez ensuite que le `Fighter other` est toujours en vie. Si ce n'est pas le cas, indiquez-le : "`nom_other` est déjà mort, ressuscitez-le afin de pouvoir le soigner." Contrôlez également qu'il a moins de 10 points de vie. Si ce n'est pas le cas, indiquez-le via le texte suivant : "`nom_other` a déjà le maximum de points de vie."

Si toutes ces conditions sont réunies, ajoutez la valeur de l'attaque de l'instance qui appelle la méthode aux points de vie de `other`, puis appelez la méthode de classe `checkHealth()`.

Conseil

Pensez à utiliser un branchement conditionnel pour les contrôles.

Le nombre de points de vie à ajouter est simplement égal à l'attaque de l'instance depuis laquelle la méthode est appelée. Ajoutez la valeur de cet attribut `attack` au `Fighter other`.

Solution

```
1  public void attack(Fighter other) {
2      if(!this.isAlive()) {
3          System.out.println(this.getName() + " est mort et ne peut plus
       rien faire");
4      }
5      else{
6          if (other.getHealth() >= 10) {
7              System.out.println(other.getName() + " a déjà le maximum
       de points de vie");
8          }
9          if (!other.isAlive()) {
10             System.out.println(other.getName() + " est déjà mort,
       ressuscitez-le pour pouvoir le soigner");
11         } else {
12             other.setHealth(other.getHealth() + this.getAttack());
13             Fighter.checkHealth();
14         }
15     }
16 }
```

Question 12: (*5 minutes*) Classe-fille `Attaquant`

Commencez par déclarer une nouvelle classe-fille `Attaquant` héritant de la classe `Fighter`. Cette classe-fille prendra un nouvel attribut privé nommé `multiplicateur` de type `int`, qui sera passé comme argument du `constructeur` de la classe-fille.

Déclarez le `constructeur` de cette classe ainsi que le `getter` et le `setter` permettant d'interagir avec ce nouvel attribut `multiplicateur`.

Conseil

Pensez à utiliser le `constructeur` de votre classe-mère `Fighter`.

Solution

```java
class Attaquant extends Fighter {

    private int multiplicateur;

    public Attaquant(String name, int health, int attack, int defense,
    int multiplicateur) {
        super(name, health, attack, defense);
        this.multiplicateur = multiplicateur;
    }

    public int getMultiplicateur() {
        return multiplicateur;
    }

    public void setMultiplicateur(int multiplicateur) {
        this.multiplicateur = multiplicateur;
    }
}
```

Question 13: (*10 minutes*) Méthode `attack` de la classe-fille `Attaquant`

Réécrivez la méthode `attack` de la classe-fille `Attaquant` afin d'effectuer plusieurs attaques sur `other` en fonction de l'attribut `multiplicateur`.

Assurez-vous de toujours indiquer le type de l'attaque lorsque vous faites appel à la méthode `attack()`.

Conseil

Pensez à contrôler que l'instance depuis laquelle la méthode est appelée est encore en vie. Aidez-vous de la méthode `attack` de la classe-mère `Fighter`.

Pour effectuer plusieurs attaques, vous pouvez utiliser une boucle allant de 0 à `multiplicateur`.

Solution

```java
public void attack(String type, Fighter other) {
    if (!this.isAlive()) {
        System.out.println(this.getName() + " est mort et ne peut plus
    rien faire");
        return;
    }
    for (int i = 0; i < this.getMultiplicateur(); i++) {
        System.out.println("Attaque n " + (i + 1));
        super.attack(type, other);
    }
}
```

Pour vérifier votre programme, exécutez la méthode `main` suivante :

```java
public class FighterMain {
    public static void main(String[] args) {
        Fighter P1 = new Fighter("P1", 10, 2, 2);
        Attaquant P2 = new Attaquant("P2", 10, 2, 2, 2);
        Soigneur P3 = new Soigneur("P3", 10, 4, 2, 4);
```

```
        P1.attack("pied", P2);
        P1.attack("poing", P2);
        P1.attack("tete", P2);
        P1.attack("tete", P2);
        P3.resurrection(P2);
        P1.attack("pied", P2);
        P1.attack("poing", P2);
        P1.attack("tete", P2);
        P3.attack(P2);
        P2.attack("tete", P1);
    }
}
```

Vous devriez obtenir :

```
P1 a encore 10 points de vie
P2 a encore 8 points de vie
P3 a encore 10 points de vie
------------------------------
P1 a encore 10 points de vie
P2 a encore 6 points de vie
P3 a encore 10 points de vie
------------------------------
P1 a encore 10 points de vie
P2 a encore 2 points de vie
P3 a encore 10 points de vie
------------------------------
P2 est mort
P1 a encore 10 points de vie
P3 a encore 10 points de vie
------------------------------
P2 vient de revenir à la vie
P1 a encore 10 points de vie
P3 a encore 10 points de vie
P2 a encore 10 points de vie
------------------------------
P1 a encore 10 points de vie
P3 a encore 10 points de vie
P2 a encore 8 points de vie
------------------------------
P1 a encore 10 points de vie
P3 a encore 10 points de vie
P2 a encore 6 points de vie
------------------------------
P1 a encore 10 points de vie
P3 a encore 10 points de vie
P2 a encore 2 points de vie
------------------------------
P1 a encore 10 points de vie
P3 a encore 10 points de vie
P2 a encore 6 points de vie
------------------------------
Attaque n 1
P1 a encore 6 points de vie
P3 a encore 10 points de vie
P2 a encore 6 points de vie
------------------------------
Attaque n 2
P1 a encore 2 points de vie
P3 a encore 10 points de vie
P2 a encore 6 points de vie
------------------------------
```

3 Notions d'héritage en Python

Question 14: (*15 minutes*) **Classe Point (Suite)**
La semaine précédente, vous avez défini une classe `Point` représentant un point de 2 dimensions, avec des coordonnées `x` et `y`, ainsi que des opérations basiques sur des points 2D. Pour cet exercice, vous allez implémenter une classe de points à 3 dimensions qui héritera de la classe `Point` créée précédemment. Avant de commencer, modifiez les attributs privés de la classe `Point`. Remplacez-les par des attributs protégés. Écrivez une classe qui hérite de `Point`. Nommez-la `Point3D`. Après avoir rajouté la troisième dimension comme attribut, effectuez les opérations ci-dessous :

- Rajoutez une méthode qui renvoie une représentation vectorielle du point. Vous pouvez utiliser une liste.
- Recalculez la distance euclidienne et le milieu pour le point 3D.
- Pour aller plus loin Si vous voulez vous familiariser encore plus avec les méthodes de classe en Python, implémentez deux autres algorithmes de calcul de distance : les distances de Manhattan et Minkowski.

Vous pouvez compléter le code suivant (qui se trouve sur Moodle, dans le dossier `Code`).

```
1  import math
2
3  class Point3D(Point):
4      def __init__(self, x, y):
5          super().__init__(x, y)
6          # Rajoutez un nouvel attribut z propre à la classe Point3D
7
8
9      # Définir le getter et le setter sur le nouvel attribut z
10
11     # Complétez les méthodes suivantes
12     def vector_representation(self): # représentée sous forme de liste
13         ...
14
15     def distance_euclidean(self, p2): # i.e norme
16         ...
17
18     def distance_manhattan(self, p2):
19         ...
20
21     def distance_minkowski(self, p2, order=3):
22         ...
23
24     def milieu(self, p2):
25         ...
```

Conseil

- En Python, pour définir un attribut protégé, on le précède d'un seul underscore (par exemple `_x = 45` est un attribut protégé).
- L'instruction `super().__init__(x, y)` permet de faire appel au constructeur de la classe-mère.
- Dans un espace à 3 dimensions, la formule pour calculer la distance entre les points $p_1 = (x_1, y_1, z_1)$ et $p_2 = (x_2, y_2, z_2)$ est $\sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2 + (z_1 - z_2)^2}$.

Solution

```python
import math
import Point

class Point3D(Point):
    def __init__(self, x, y, z):
        super().__init__(x, y)
        self._z = z

    def get_z(self):
        return self._z

    def set_z(self, z):
        self._z = z

    def vector_representation(self): # représentée sous forme de liste
        return [self._x, self._y, self._z]

    def distance_euclidean(self, p2): # i.e norme
        other_x = p2.get_x()
        other_y = p2.get_y()
        other_z = p2.get_z()
        return math.sqrt((self._x - other_x)**2 + (self._y - other_y)**2 + (self._z - other_z)**2)

    def distance_manhattan(self, p2):
        other_x = p2.get_x()
        other_y = p2.get_y()
        other_z = p2.get_z()
        return sum([abs(self._x - other_x), abs(self._y - other_y), abs(self._z - other_z)])

    def distance_minkowski(self, p2, order=3):
        other_x = p2.get_x()
        other_y = p2.get_y()
        other_z = p2.get_z()
        return sum([abs(self._x - other_x)**order, abs(self._y - other_y)**order, abs(self._z - other_z)**order])**(1/order)

    def milieu(self, p2):
        other_x = p2.get_x()
        other_y = p2.get_y()
        other_z = p2.get_z()

        x_M = (self._x + other_x)/2
        y_M = (self._y + other_y)/2
        z_M = (self._z + other_z)/2
        return Point3D(x_M, y_M, z_M) # renvoie un point!


point1 = Point3D(1, 2, 3)
point2 = Point3D(3, 4, 5)

# exemple
point1.vector_representation()
```